Unit 2: Introduction to the Transport Layer

The Transport Layer Service Model

The transport layer provides the foundational service for data communication directly between applications. To facilitate this, the transport layer utilizes distinct protocols that offer varying levels of reliability and abstraction, which will be deeply analyzed in this unit.

Protocol	Service Model	Key Characteristics
TCP (Transmission Control Protocol)	Reliable, bidirectional byte stream.	Handles formal connection setups and ensures data transfers correctly. *(Note: Advanced high-performance tuning for TCP is reserved for later units; this focus is strictly on correct operation)*.
UDP (User Datagram Protocol)	Simple, unreliable datagrams.	Provides fast, connectionless transmission without built-in delivery guarantees.
ICMP (Internet Control Message Protocol)	Control and error notification.	Although often associated closely with the network layer, this protocol is utilized to carry critical control information—such as routing error notifications—across the Internet.

> **Crucial Callout: The End-to-End Argument.** A newly introduced architectural concept in this unit is the "end-to-end argument" (or "end-to-end principle"). This principle acts as the governing rule for network engineers, dictating exactly *how* and *where* a network feature should be implemented to guarantee it functions correctly.

Error Detection Algorithms

For TCP to reliably transfer data, the protocol must have a mathematical way to know if a payload has been corrupted during transit across physical links. Network engineers utilize three primary algorithms to detect these errors:

- **Checksums:** Mathematical sums (often utilizing $1s$ complement arithmetic) calculated over the packet to detect bit-level corruption.
- **Cyclic Redundancy Checks (CRCs):** Advanced polynomial division algorithms used to detect accidental changes to raw data, highly effective against burst errors.

- **Message Authentication Codes (MACs):** Cryptographic checksums that verify both the data's integrity and its authenticity.

Protocol Design: Finite State Machines

To systematically answer complex questions like "How exactly does TCP set up a connection?" or "What do TCP segments look like?", network engineers rely on a specific modeling tool.

1. **Finite State Machines (FSMs):** A basic mathematical approach and tool utilized universally in network protocol design.
2. **State Tracking:** Because TCP is a reliable protocol, it must constantly track the "state" of the connection (e.g., listening, synchronizing, established, closed).
3. **TCP State Walkthrough:** This unit will explore the exact FSM that TCP uses to seamlessly transition between these connection states, mapping out the precise lifecycle of a reliable data transfer.

The TCP Service Model

Introduction to the Transmission Control Protocol (TCP)

The Transmission Control Protocol (TCP) is utilized by over 95% of modern Internet applications. It is universally adopted because it provides a highly reliable, end-to-end, bidirectional byte-stream service that abstractly acts as a continuous pipe between two application processes.

When two applications communicate via TCP, they establish a formal, two-way communication channel (a "connection") between their respective peer TCP layers.

The TCP Service Properties

TCP provides several distinct services to the application layer and the network at large, ensuring data arrives flawlessly regardless of underlying network conditions.

Service Property	Operational Behavior	Core Mechanisms
Stream of Bytes	Delivers a continuous byte stream.	Emulates a continuous flow by breaking application data into discrete **TCP Segments** (which can range from 1 byte, like a keystroke, up to the maximum IP datagram size).
Reliable Delivery	Guarantees data is delivered correctly.	1. **Acknowledgments** confirm receipt. 2. **Checksums** detect corrupted bits. 3. **Sequence Numbers** track missing data.

Service Property	Operational Behavior	Core Mechanisms
		4. **Flow-Control** prevents overwhelming the receiver.
In-Sequence Delivery	Reorders packets upon arrival.	Ensures data is handed to the receiving application in the exact order it was originally transmitted, utilizing Sequence Numbers to re-sequence out-of-order packets.
Congestion Control	Protects overall network health.	Actively attempts to divide up available network capacity equally among all active TCP connections.

> **Crucial Callout:** TCP handles Flow-Control by having the receiver constantly broadcast how much room it has left in its buffers. If a receiver falls behind, it will advertise a shrinking buffer space (potentially down to zero), forcing the sender to halt transmission until space clears up.

Chronological Flow 1: Connection Setup (The 3-Way Handshake)

Before any application data can be transferred, the TCP layers must formally establish the connection and agree on starting parameters.

1. **SYN (Synchronize):** Host A initiates the connection by sending a `SYN` message to Host B. This packet includes Host A's Initial Sequence Number (N), which dictates the base number Host A will use to identify bytes in its stream.
2. **SYN + ACK (Synchronize & Acknowledge):** Host B receives the request and replies with a single packet that serves two purposes. It asserts an `ACK` flag to acknowledge Host A's request, and it asserts a `SYN` flag to establish the reverse channel, including its own distinct Initial Sequence Number (N).
3. **ACK (Acknowledge):** Host A replies with a final `ACK` to acknowledge Host B's sequence number. The connection is now established in both directions, and data transfer may begin.

Chronological Flow 2: Connection Teardown

When the applications finish transmitting data, TCP must gracefully close the connection and delete the state machine data from memory.

1. **FIN (Finish):** Host A sends a `FIN` message to Host B, indicating it has no more data to send.
2. **ACK (+ Data):** Host B sends an `ACK` to confirm Host A's stream is closed. At this point, the A $\rightarrow$ B channel is closed, but the B $\rightarrow$ A

channel remains open. Host B may continue sending data to Host A indefinitely if needed.

3. **FIN (Finish):** Once Host B completes its own data transmission, it sends its own `FIN` message to Host A.
4. **ACK (Acknowledge):** Host A replies with a final `ACK`. The connection is now fully terminated in both directions.

The TCP Segment Header Architecture

Because TCP is reliable, its header is much longer and more complex than an Ethernet or IP header, requiring extensive metadata to track connection status.

Header Field	Function & Behavior
Destination Port	Tells the receiving TCP layer which specific application process to route the data to (e.g., Port `80` for a Web Server, Port `22` for SSH).
Source Port	A locally generated identifier that tells the receiving host what port to use when sending replies back.
Sequence Number	Indicates the position in the overall byte stream of the *first* byte contained within this specific segment.
Acknowledgment Number	Indicates the *next* byte sequence number the host is expecting to receive, simultaneously confirming that all bytes prior to this number have been successfully received.
Header Length (HLEN)	Specifies the total length of the TCP header, which fluctuates if TCP Options are present.
Checksum	A 16-bit mathematical sum calculated over the entire header and data payload to detect hardware-level bit corruption.

TCP Control Flags:

- **ACK:** Validates that the Acknowledgment Number field is active and accurate.
- **SYN:** Signals a synchronization request during the 3-way handshake.
- **FIN:** Signals the termination of one direction of the connection.
- **PSH (Push):** Instructs the receiving TCP layer to immediately deliver the data to the application rather than waiting to buffer more data (highly useful for real-time keystrokes).

Connection Uniqueness and Collision Prevention

A TCP connection is uniquely identified across the entire global Internet by a strict 5-tuple derived from the IP and TCP headers:

1. Source IP Address
2. Destination IP Address
3. Protocol ID (which is "TCP")
4. Source Port
5. Destination Port

To prevent a new connection from accidentally overlapping with an older, delayed connection (e.g., if an old packet gets stuck in a router buffer and arrives late), TCP employs two preventative measures:

- **Sequential Port Generation:** The initiating host strictly increments the source port number for every single new connection (wrapping around only after 64k connections).
- **Randomized Initial Sequence Numbers (ISNs):** Instead of starting every byte stream at byte `0`, TCP initializes connections with a completely random ISN.

The UDP Service Model

Introduction to the User Datagram Protocol (UDP)

The User Datagram Protocol (UDP) is the second major transport layer protocol. It is utilized specifically by applications that either do not require guaranteed data delivery or prefer to handle retransmissions using their own private, custom logic. UDP is fundamentally much simpler than TCP; its primary function is merely taking application data, creating a UDP datagram, and handing it to the network layer.

The UDP Datagram Format

Because the service it offers is intentionally bare-bones, the UDP header is extremely small. While TCP utilizes over ten header fields, UDP utilizes just four.

Header Field	Size	Function & Behavior
Source Port	16-bit	Indicates which specific application process the data originated from, allowing the remote host to direct any replies back correctly.
Destination Port	16-bit	Directs the incoming packet to the correct application process on the receiving end-host.
Length	16-bit	Specifies the total length of the entire UDP datagram (header plus data payload) in bytes. Because the UDP header is exactly 8 bytes long, the absolute minimum value for this field is 8.

Header Field	Size	Function & Behavior
Checksum	16-bit	An optional error-checking mechanism when running over IPv4. If the sender chooses not to compute a checksum, this field is filled entirely with zeroes.

> **Crucial Callout: The Pseudo-Header Violation.** If a UDP checksum is utilized, the calculation covers the UDP header and the UDP data, but it *also* includes a portion of the IPv4 header (specifically the IP Source Address, IP Destination Address, and the Protocol ID of 17). This is an intentional architectural violation of the strict 4-layer model (where a layer should only care about its own data). This violation occurs to allow the UDP layer to detect if a datagram was accidentally delivered to the wrong IP destination.

UDP Port Demultiplexing

It is highly useful to conceptualize UDP not as a complex transport protocol, but merely as a demultiplexing mechanism designed to divide up an incoming stream of datagrams and route them to the correct application processes. (For this reason, some engineers joke that UDP stands for "User Demultiplexing Protocol").

1. Process 1 on Host A generates data intended for Process 1 on Host B (which uses Port `177`).
2. Host A encapsulates the data in a UDP datagram with the Destination Port set to `177` and includes its own Source Port.
3. The UDP datagram is encapsulated inside an IP datagram and transmitted to Host B.
4. Host B receives the IP datagram, extracts the UDP datagram, reads the port number (`177`), and routes the data directly to Process 1.

The UDP Service Model Properties

The UDP service model is essentially just a lightweight wrapper placed on top of the underlying, unreliable IP layer.

Service Property	Behavioral Definition
Connectionless Datagram Service	No connection is formally established before transmission. Because every single packet is entirely self-contained, packets may arrive completely out of order.
Unreliable Delivery	UDP makes absolutely zero delivery guarantees. It does not send acknowledgments, does not detect missing data, and does not provide flow-control to prevent overwhelming a slow receiver.

Why Use UDP? (Common Use Cases)

Despite its unreliability, UDP is highly favored for specific network tasks where speed and simplicity are prioritized over guaranteed delivery.

- **Simple Request-Reply Systems:** Applications that rely on a single, quick request and a single, short response.
  - **DNS (Domain Name System):** Resolving hostnames to IP addresses.
  - **DHCP (Dynamic Host Configuration Protocol):** Used by computers to request local IP address assignments from a router.
- **Time-Sensitive Media:** Real-time applications where waiting for a dropped packet to be retransmitted would cause unacceptable lag. While historically used for video conferencing or streaming, it is worth noting that much of this traffic has now migrated to TCP-based protocols (like HTTPS streaming).

The Internet Control Message Protocol (ICMP)

Introduction to Network Layer Operations

Making the Network Layer function correctly requires the coordinated effort of three distinct operational components:

1. **The Internet Protocol (IP):** Responsible for the creation of IP datagrams and the hop-by-hop delivery of data from end to end.
2. **Routing Tables:** Built by algorithms to populate router forwarding tables, dictating the physical path packets must take.
3. **The Internet Control Message Protocol (ICMP):** Responsible for communicating critical network layer information between end-hosts and routers, reporting error conditions, and diagnosing network problems.

The ICMP Service Model

ICMP acts as the diagnostic nervous system of the Internet. Its service model is defined by two primary characteristics:

Service Property	Operational Behavior
Reporting Message	Functions as a completely self-contained message designed specifically to report a network error or condition.
Unreliable	Operates as a simple, unacknowledged datagram service. If an ICMP message is lost in transit, the network performs absolutely no retries to deliver it.

Architectural Position and Encapsulation

While ICMP exists exclusively to communicate information *about* the Network Layer, it technically runs *above* the Network Layer in the protocol stack.

The Encapsulation Flow:

1. The network generates an **ICMP Message** (containing an ICMP header and error data).
2. This ICMP Message is passed down to the Network Layer, where it becomes the data payload inside a standard **IP Datagram**.
3. The IP Datagram is passed down to the Link Layer, where it is encapsulated inside a **Link Frame** for physical transmission.

> **Crucial Callout:** Because ICMP messages are encapsulated directly inside IP datagrams (sitting logically above the IP layer), network architects strictly classify ICMP as a **Transport Layer mechanism**, despite its purpose of diagnosing Network Layer routing issues.

ICMP Message Types and Codes (RFC 792)

ICMP identifies specific network conditions using a combination of a `Type` value and a `Code` value embedded in its header.

ICMP Type	ICMP Code	Standard Description	Common Use Case / Meaning
0	**0**	Echo Reply	The successful response to a `ping` command.
3	**0**	Destination Network Unreachable	A router cannot locate a route to the target IP's broader network.
3	**1**	Destination Host Unreachable	A router reached the destination network, but the specific end-host is offline or unlocatable.
8	**0**	Echo Request	The outbound probe initiated by a `ping` command.
11	**0**	Time to Live (TTL) Exceeded	Used by `traceroute` to map network hops when a packet's loop-prevention counter hits zero.

(Note: The provided source material specifically highlights Types 0, 3, 8, and 11 as the primary diagnostic codes.)

Network Diagnostic Tools Utilizing ICMP

Two of the most universally deployed network troubleshooting tools rely entirely on the ICMP service model.

1. The `ping` Utility (Testing Connectivity) The `ping` command verifies if a remote host is online and reachable.

1. Host A generates an ICMP **Type 8, Code 0** (Echo Request) message and sends it to Host B.
2. Host B receives the request and immediately generates an ICMP **Type 0, Code 0** (Echo Reply) message.
3. Host B transmits the reply back to Host A, proving bi-directional connectivity.

2. The `traceroute` Utility (Mapping Network Paths) The `traceroute` command maps the exact router-by-router path a packet takes to reach its destination.

1. Host A sends a standard packet toward Host B, but artificially restricts the Time to Live (TTL) hop-counter in the IP header to 1.
2. The very first router on the path receives the packet, decrements the TTL to 0, and drops the packet.
3. Because the packet was dropped, the router generates an ICMP **Type 11, Code 0** (TTL Expired) message and sends it back to Host A.
4. Host A records the IP address of that router, then sends a second packet with a TTL of 2, forcing the *second* router on the path to drop it and return an ICMP TTL Expired message. This loop continues until the destination is reached.

The End-to-End Principle

The end-to-end principle is a foundational architectural concept in the design of the Internet that addresses two distinct but related areas: system correctness and system design philosophy.

1. Correctness and the End-to-End Argument

The original end-to-end argument focuses on ensuring that functions within a communication system are implemented correctly.

- **The Argument:** The end-to-end argument states that a function can only be completely and correctly implemented with the knowledge and help of the applications residing at the communication system's end points.
- **The Implication:** Because only the end points possess the necessary information, it is impossible to correctly implement such functions as features of the communication network itself.
- **Performance vs. Correctness:** While the network cannot be responsible for correctness, an incomplete version of a function provided by the network may still be useful as a performance enhancement.

Case Study: File Transfer Corruption In early network design, some developers assumed that since individual links provided error detection, the entire file transfer would be inherently protected. This assumption was proven false when internal router memory became corrupted; because the router checked packets *before* they were stored, the data was corrupted while in memory, passed the router's error check, and was forwarded as "valid" data.

- **Conclusion:** The only way to guarantee a file arrives uncorrupted is to perform an end-to-end check at the destination (e.g., using a hash, as in BitTorrent) after the file is fully reassembled.
- **TCP Reliability:** While TCP is designed to be a reliable byte stream, it is not perfect. Errors can still occur due to bugs in the TCP stack or other faults, making end-to-end verification a necessary practice for sensitive data transfers.

2. Performance Enhancements (Link-Layer Help)

While end-to-end functionality is required for correctness, the network can assist with performance.

- **Wireless Link Reliability:** Wireless links are significantly less reliable than wired links. To improve performance, wireless link layers often use local retransmissions (link-layer acknowledgments) to boost reliability from potentially 50-80% up to 99% or higher.
- **Outcome:** This link-layer help allows TCP—which struggles with high loss rates—to perform significantly better without violating the end-to-end principle, because the *correctness* of the data is still verified end-to-end by TCP itself.

3. The "Strong" End-to-End Principle

A broader, more general version of this principle is known as the "strong" end-to-end principle, as defined in RFC 1958.

- **The Definition:** The network's sole job is to transmit datagrams as flexibly and efficiently as possible; all other functionality should be placed at the fringes (the end points).
- **Motivation:** The goal is to maximize flexibility and simplicity.
- **The Downside of Middle-Network Optimizations:** When the network implements features to "help" end points, it makes assumptions about what those end points need. If those assumptions are wrong, it can impede innovation and make the network design "calcified" and difficult to change.

> **Crucial Callout:** There is an ongoing tension in network engineering between short-term performance gains and long-term evolvability. Network operators often implement middle-network optimizations to

> improve current performance, but doing so frequently makes it harder to redesign or evolve the network layers in the future.

Error Detection

To maintain data integrity despite potential hardware failures, such as memory corruption in routers, networks employ three primary error detection algorithms, each with distinct characteristics.

Error Detection Algorithms

Algorithm	Mechanism	Primary Use Case	Robustness
Checksum	Sums all 16-bit words in a packet.	IP, TCP	Fast and cheap to compute, but weak; can be fooled if two bit errors cancel each other out.
Cyclic Redundancy Code (CRC)	Computes the remainder of a polynomial division.	Ethernet, other link layers	Robust; stronger than checksums; detects odd numbers of bit errors, 2-bit errors, and bursts c bits long.
Message Authentication Code (MAC)	Cryptographic transformation involving a secret key.	TLS (HTTPS)	Excellent protection against malicious modification, but not designed for simple error detection.

1. Checksums

Checksums are favored for their speed and ease of computation, even in software. They typically use one's complement arithmetic:

- The checksum field is initialized to zero.
- All 16-bit words are summed, with any carry bits added back into the total.
- The bits of the final sum are flipped to create the checksum.
- While checksums reliably detect single-bit errors, they are vulnerable to multi-bit errors where changes cancel out.

2. Cyclic Redundancy Codes (CRCs)

CRCs distill n bits of data into c check bits (c n) using polynomial long division.

- **Mathematical Basis:** The message is treated as a polynomial , and it is divided by a predefined generator polynomial . The resulting remainder is the CRC.
- **Hardware Efficiency:** CRCs are easy to implement in hardware and can be computed incrementally as data is read or written, making them ideal for high-speed link layers.
- **Strong Guarantees:** A c-bit CRC guarantees detection of any single-bit error, any 2-bit error, and any single burst of errors less than or equal to c bits long.

3. Message Authentication Codes (MACs)

MACs are fundamentally security mechanisms, not error detection tools.

- **Mechanism:** Two parties share a secret s; the sender generates $c = s$.
- **Adversary Protection:** If an adversary does not have the secret s, it is computationally difficult to generate a valid c for a modified message .
- **Weakness for Error Detection:** MACs do not guarantee error detection; a single-bit flip could potentially result in the same MAC. Therefore, they are not as robust as CRCs for catching accidental transmission errors.

End-to-End Principles in Error Detection

Error detection is a classic application of the **end-to-end principle**. Because errors can occur anywhere—including within a router's internal memory after a link-layer check has passed—every layer must perform its own verification.

> **Important Note:** A failure to perform end-to-end checks can lead to significant data loss. For example, early developers at MIT once relied solely on link-layer error detection. Because those checks occurred *before* data was moved into a router's faulty main memory, the data was corrupted in storage, passed subsequent checks, and resulted in the loss of significant source code.

Consequently, modern networks utilize multiple layers of defense: the link layer uses CRCs, IP and TCP use checksums, and applications (like BitTorrent) often use their own end-to-end hashes to ensure the final data is pristine.

Finite State Machines (FSMs)

Finite State Machines (FSMs) are a standard, mathematically precise method for specifying the behavior of networked systems and protocols. An FSM consists of a **finite set of states**, representing the various configurations a system can occupy at any given time.

FSM Components and Diagramming

- **States:** Represent the configuration of the system (e.g., `LISTEN`, `ESTABLISHED`, `CLOSED`).
- **Transitions (Edges):** Define how the system moves between states. Each edge must specify:
  - **The Event:** The stimulus or input that triggers the transition.
  - **The Action:** (Optional) The specific operation the system performs during the transition.
- **Transition Rules:** For any given state, every possible event must have a unique transition. If an event occurs for which no transition is defined, the system's behavior is considered undefined.
- **Specification Design:** While it is ideal to specify every event for every state, doing so leads to overly complex diagrams. Often, engineers specify only the common cases for clarity, leaving edge cases to supporting documentation or leaving them intentionally undefined to allow for future protocol evolution.

The TCP Finite State Machine

The TCP FSM is a critical, highly detailed specification that governs how connections are reliably set up, maintained, and torn down. It comprises 12 distinct states.

1. Connection Establishment (3-Way Handshake)

The establishment phase involves both an active opener (the client) and a passive opener (the server).

Step	Initiator	Event	Action/Transition
Initial	Server	Call `listen()`	Transitions `CLOSED` → `LISTEN`.
1	Client	Call `connect()`	Client sends `SYN`; transitions `CLOSED` → `SYN_SENT`.
2	Server	Receive `SYN`	Server sends `SYN/ACK`; transitions `LISTEN` → `SYN_RECEIVED`.
3	Client	Receive `SYN/ACK`	Client sends `ACK`; transitions `SYN_SENT` → `ESTABLISHED`.
Final	Server	Receive `ACK`	Server transitions `SYN_RECEIVED` → `ESTABLISHED`.

> **Crucial Callout:** To prevent connection ID collisions (where old, delayed packets interfere with new connections), Host A increments its source port for every new connection and initializes connections with a **randomized initial sequence number (ISN)**.

2. Data Exchange

- **State:** `ESTABLISHED`.
- **Function:** This is the center of the FSM where data is actively sent and received between the two peers.

3. Connection Teardown

TCP connection teardown is inherently more complex than establishment because TCP connections are bidirectional; one side may finish sending data while the other continues.

- **Active Closer:** The party that calls `close()` first, transitioning to `FIN_WAIT_1` and sending a `FIN` (Finish) packet.
- **Passive Closer:** The party that receives the `FIN`, transitioning to `CLOSE_WAIT` and sending its own `ACK`.
- **Closing States:**
  - **FIN_WAIT_1 / FIN_WAIT_2:** States for the active closer waiting for acknowledgments and the peer's `FIN`.
  - **CLOSING / TIME_WAIT:** States used to ensure all packets are drained from the network before returning to the `CLOSED` state.
  - **LAST_ACK:** The final state for the passive closer after it has sent its own `FIN`.

System initialized. I am ready to process your cybersecurity and network materials. Please paste your next batch of lecture slides, notes, or PDF texts whenever you are ready.

Flow Control: The Sliding Window Protocol

To improve network utilization beyond the inefficient "Stop-and-Wait" method, transport protocols employ **Sliding Window** flow control.

1. Limitations of Stop-and-Wait

- **Mechanism:** The sender transmits exactly one packet and must wait for an acknowledgment (ACK) before sending the next one.
- **Inefficiency:** On high-bandwidth, long-delay paths (e.g., San Francisco to Boston), this keeps the "pipe" largely empty, as the sender spends most of its time idle while waiting for round-trip acknowledgments.
- **Requirement:** A 1-bit counter is sufficient to detect duplicate packets.

2. Sliding Window Mechanism

- **Generalization:** Allows multiple unacknowledged segments to be "in flight" at the same time, effectively keeping the network pipe full.

- **Window Size:** The maximum number of unacknowledged segments allowed in flight is defined as the window size.

Sender Side

- **Variables:**
  - **SWS (Send Window Size):** The maximum number of segments allowed in flight.
  - **LAR (Last Acknowledgment Received):** The sequence number of the most recently acknowledged segment.
  - **LSS (Last Segment Sent):** The sequence number of the most recent segment transmitted.
- **Invariant:** The sender must ensure $L - L$.
- **Behavior:** The sender advances the window (LAR) as new ACKs arrive and buffers up to segments.

Receiver Side

- **Variables:**
  - **RWS (Receive Window Size):** The maximum buffer size for incoming segments.
  - **LSR (Last Segment Received):** The sequence number of the last segment successfully processed.
  - **LAS (Last Acceptable Segment):** The highest sequence number the receiver is willing to accept.
- **Invariant:** $L - L$.
- **Behavior:** If a packet is received, the receiver sends cumulative ACKs (e.g., if packets 1, 2, 3, and 5 are received, the receiver acknowledges 3).
  - *Note:* TCP acknowledgments specifically indicate the *next* expected data sequence number (e.g., ACK 4).

3. Sequence Number Space Requirements

The necessary size of the sequence number space is determined by the window sizes.

- **General Rule:** Generally, the sequence number space must be at least $+$.
- **Go-Back-N Protocol ($= 1$):** Requires $+1$ sequence numbers.
- **Symmetric Windows ($=$):** Requires $2 \times$ sequence numbers.

4. TCP Flow Control

- **Implementation:** The receiver explicitly advertises its remaining buffer capacity to the sender using the "window" field in the TCP header.

- **Sender Constraint:** The sender is restricted from transmitting data beyond $L + wndow$.
- **Purpose:** This mechanism prevents a fast sender from overrunning the memory buffers of a slower receiver.

Flow Control: Stop-and-Wait

Flow control ensures that a sender does not transmit more data than a receiver can successfully process. To manage this, the receiver must provide feedback to the sender regarding its capacity. The **Stop-and-Wait** protocol is one of the two foundational approaches to this problem.

1. Stop-and-Wait Protocol Basics

- **Packet Limitation:** At any given moment, there is at most one packet in flight between the sender and receiver.
- **Operational Cycle:**
  1. The sender transmits a single packet.
  2. The receiver processes the data and sends an acknowledgment (ACK) packet back.
  3. Upon receiving the ACK, the sender is cleared to transmit the next packet.
- **Timeout Mechanism:** If an acknowledgment is not received within a specified time, the sender assumes the data was lost and triggers a retransmission of the current packet.

2. Finite State Machine (FSM) Representation

The protocol logic is defined using Finite State Machines (FSMs) for both the sender and the receiver.

- **Receiver FSM:**
  - **State:** The receiver remains in the "Wait for Packets" state.
  - **Action:** When it receives data, it delivers the data to the application and sends an ACK packet back to the sender.
- **Sender FSM:**
  - **State:** The sender alternates between "Wait for Data" and "Wait for ACK" states.
  - **Action:** When software calls the send function, the sender transmits the packet and moves to the "Wait for ACK" state.
  - **Exception:** If a timeout occurs while waiting for an ACK, the sender re-executes the send action.

3. Handling Ambiguity and Duplicates

Because networks are imperfect and can lose or delay packets, Stop-and-Wait must prevent the receiver from confusing retransmitted packets with

new data.

- **1-Bit Counter:** A 1-bit counter is included in both data packets and acknowledgment packets.
- **Duplicate Detection:** This allows the receiver to determine if an arriving packet is a new piece of data or a redundant duplicate that should be ignored.
- **Assumptions:** This logic assumes that the underlying network does not maliciously duplicate packets and that packets are not delayed long enough to trigger multiple timeouts.

4. Example Execution Scenarios

The protocol must be robust enough to handle various network-level failures:

Scenario	Protocol Behavior
No Loss	Data reaches the receiver; ACK reaches the sender; the cycle continues efficiently.
Data Loss	Data is dropped in transit; the sender's timer expires; the sender retransmits the same data packet.
ACK Loss	The receiver gets the data and sends an ACK, but the ACK is dropped; the sender times out and retransmits; the receiver uses the 1-bit counter to detect and discard the duplicate, then resends the ACK.
ACK Delay	The ACK is delayed beyond the sender's timeout; the sender retransmits; the receiver handles the duplicate via the 1-bit counter.

TCP Header Anatomy

The TCP header is essential for providing a reliable, bidirectional byte-stream service. It is 32 bits (4 octets) wide, and its structure enables the tracking of connections, sequence management, and error detection.

- **Source Port:** Identifies the port number of the application originating the data.
- **Destination Port:** Identifies the port number of the application process at the receiving host.
- **Sequence Number:** Indicates the position in the byte stream of the first byte within the TCP data field.
- **Acknowledgment Number:** Specifies the sequence number of the next byte expected by the receiver, effectively acknowledging receipt of all preceding bytes.

- **Offset:** Indicates the length of the TCP header, which is necessary as the header length can vary when optional fields are included.
- **Reserved:** Reserved for future use.
- **Flags (UAPRSF):** Control flags used to signal connection status and data handling:
  - **U (Urgent):** Indicates the urgent pointer field is valid.
  - **A (ACK):** Indicates the acknowledgment number field is valid.
  - **P (PSH):** Requests that the TCP layer deliver data to the application immediately.
  - **R (RST):** Resets the connection.
  - **S (SYN):** Synchronizes sequence numbers during connection setup.
  - **F (FIN):** Signals that the sender has finished transmitting data.
- **Window:** Specifies the size of the receive buffer, used by the receiver for flow control to manage how much data the sender can transmit.
- **Checksum:** Used to detect corruption in the TCP header and data payload.
- **Urgent Pointer:** Points to the end of urgent data within the segment.
- **Options:** Provides a space for additional header fields that were added after the original TCP standard.
- **Padding:** Used to ensure the header ends on a 32-bit boundary.

TCP Setup and Teardown

Reliable communication in TCP requires maintaining distinct connection states at both end-hosts. The protocol manages the lifecycle of these connections through structured establishment and teardown phases, utilizing the control flags within the TCP header.

1. Connection Establishment: The 3-Way Handshake

To set up a connection, TCP uses a 3-way handshake to synchronize state and sequence numbers between an active opener (client) and a passive opener (server).

- **SYN (Synchronize):** The active opener sends the first packet containing a `SYN` flag and its initial sequence number ().
- **SYN + ACK:** The passive opener responds with a `SYN` flag (its own initial sequence number,) and an `ACK` flag acknowledging the active opener's sequence number.
- **ACK:** The active opener responds with an `ACK` to acknowledge the passive opener's sequence number, completing the handshake.

> **Note:** TCP also supports a "simultaneous open" scenario where two hosts send `SYN` packets to each other at the same time; each side

> acknowledges the other's request to successfully establish the connection.

2. Connection Teardown

Terminating a bidirectional TCP connection requires both sides to agree that transmission is complete.

- **FIN Bit:** The `FIN` (Finish) flag indicates that the sender has no more data to send. This is typically triggered by a `close()` or `shutdown()` system call.
- **Teardown Exchange:**
  1. **A** $\rightarrow$ **B:** Send `FIN`, sequence , acknowledge .
  2. **B** $\rightarrow$ **A:** Send `ACK` for $_{+1}$.
  3. **B** $\rightarrow$ **A:** Send `FIN`, sequence , acknowledge $_{+1}$.
  4. **A** $\rightarrow$ **B:** Send `ACK` for $_{+1}$.

3. Safety and Cleanup (TIME_WAIT)

Closing a socket creates potential hazards, such as the loss of the final `ACK` or the immediate reuse of a port pair for a new, conflicting connection.

- **TIME_WAIT State:** The "active" closer (the side that initiates the `FIN`) enters a `TIME_WAIT` state after the final exchange.
- **2MSL Rule:** The socket is held for $2 \times L$ (Maximum Segment Lifetime) to ensure that any lingering packets in the network are discarded and do not interfere with subsequent connections.
- **Practical Implications:**
  - Servers with high connection volumes may experience slowdowns if too many sockets remain in `TIME_WAIT`.
  - **SO_LINGER:** An option to send a `RST` (Reset) packet to immediately delete the socket, though this is a "hack" that bypasses standard cleanup.
  - **SO_REUSEADDR:** A socket option that allows a server to restart and re-bind to a port number that is still in use by a socket in `TIME_WAIT`.

Summary of TCP Header Flags

The TCP header utilizes six flags (bits in the `UAPRSF` field) to manage connection transitions:

- **U (Urgent):** Indicates urgent data is present.
- **A (ACK):** Acknowledgment number is valid.
- **P (PSH):** Push function; requests immediate data delivery.
- **R (RST):** Reset the connection.

- **S (SYN):** Synchronize sequence numbers.
- **F (FIN):** Finish; no more data from the sender.

Retransmission Strategies

Reliable transport requires mechanisms to handle packet loss or corruption, ensuring that all data is correctly delivered to the application. Protocols with a window of packets in flight use cumulative acknowledgments (ACKs) and per-packet timers to manage this process.

1. Go-Back-N (GBN)

Go-Back-N is a retransmission strategy where the receiver's window size is effectively 1.

- **Behavior on Loss:** If a single packet is lost, the receiver does not buffer out-of-order packets. Consequently, the sender must retransmit the lost packet *and* all subsequent packets in the current window.
- **Performance:** This is considered a "pessimistic" approach because a single loss event results in the retransmission of the entire window.

2. Selective Repeat (SR)

Selective Repeat is a more efficient strategy where the receiver is capable of buffering out-of-order packets.

- **Behavior on Loss:** If a single packet is lost, the receiver buffers subsequent packets that arrive correctly. The sender only needs to retransmit the specific packet that was lost.
- **Performance:** This is considered an "optimistic" and more efficient approach, as it minimizes redundant transmissions.

Summary of Protocol Behavior

The retransmission strategy can often be inferred by observing the sender and receiver window sizes in a protocol exchange:

Protocol Strategy	Sender Window (N)	Receiver Window	Key Characteristic
Go-Back-N	N	1	A single loss causes the entire window to be retransmitted.
Selective Repeat	N	N	Only the missing packet is retransmitted; out-of-order packets are buffered.

> **Crucial Callout:** The choice between these two strategies significantly impacts network efficiency. Go-Back-N is simpler to implement because the receiver does not need to store out-of-order segments, but

> Selective Repeat offers superior performance in lossy environments by avoiding unnecessary retransmissions.

Summary & Review

This unit focused on the Transport Layer, which provides the critical service for data communication between applications. The instruction covered the primary transport layer protocols and the overarching architectural principle of the Internet.

Summary of Transport Layers

Three main transport layers were analyzed, each serving distinct application needs:

Protocol	Service Model	Key Use Cases / Behavior
TCP	Reliable, bidirectional byte stream.	Used by >95% of Internet applications; provides end-to-end delivery guarantees.
UDP	Simple, unreliable datagram service.	Used by applications that do not need reliable delivery or prefer private retransmission logic (e.g., DNS, DHCP).
ICMP	Network feedback and error reporting.	Used to report when network layer communications fail (e.g., destination unreachable) and to monitor route performance.

TCP Mechanics and Design

The unit provided an in-depth exploration of how TCP maintains reliable delivery across an inherently unreliable Internet:

- **Reliability Mechanisms:** Exploration of error detection (corruption) and retransmission strategies for missing packets.
- **Retransmission Strategies:** Detailed study of strategies including "Selective Repeat" and "Go-Back-N".
- **Sliding Window:** Analysis of how TCP tracks outstanding, unacknowledged bytes to maintain efficient throughput.
- **Connection Management:** Examination of how connections are established, maintained, and torn down using the TCP finite state machine.

The End-to-End Principle

This principle remains a vital guide for architectural design in the Internet and other communication systems.

- **The Milder Version:** States that certain functions (such as end-to-end security or reliable file transfer) can *only* be correctly implemented at the edges (the "fringe") of the network. While the network can provide functions to *help* these features, it cannot replace the end-to-end functionality.
- **The Stronger Version:** States that if a function *can* be implemented at the end hosts, it *should* be. This approach keeps the network core simple, streamlined, and easier to maintain, while leveraging the intelligence (e.g., processing power) of modern end hosts like smartphones and laptops.